

Forecasting Average Traffic Speed on the PEMS08 Sensor Network

Near-future and long-term traffic speed forecasting

Vita Naglič Matevž Rom Nace Kovačič

Artificial Intelligence

Traffic speed forecasting is a graph time-series problem

Question

Given recent measurements from freeway sensors, how fast will traffic move later at each sensor?

This project

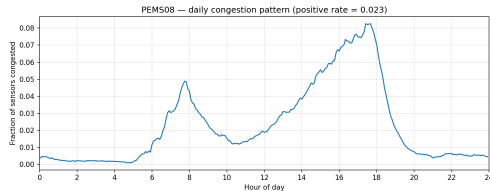
We compare neural models for two forecasting ranges:

- near future: +5 to +60 minutes
- longer term: +1h, +3h, +6h, +12h

Main message

PEMS08 is a node-level regression task on a road graph: each sensor has its own time series, and neighbouring sensors provide traffic context.

Speed forecasts support travel-time estimation, congestion warnings, and routing decisions.



Congestion follows daily structure, but is rare overall: 2.27% of sensor-time points.

PEMS08 gives us a sparse but connected road graph

PEMS08 sensor network — spring layout



PEMS08 benchmark

sensors	170
time steps	17,856
resolution	5 min
duration	62 days

Road graph facts

- 277 unique directed edges
- one weakly connected component
- average in/out-degree 1.63
- 16 Louvain communities

Modeling consequence

Each sensor has its own time series, but nearby sensors provide useful context.

NN1 – what goes in and what comes out

Task: for each sensor, predict its speed one hour from now.

Input (what the model sees): the last hour of data from every sensor – 12 readings, one every 5 minutes.

Output (what it predicts): one number per sensor – the speed in 60 minutes.

Each reading gives 10 numbers per sensor:

- the 3 measured signals: flow, occupancy, speed;
- the time of day and day of week (so it learns rush-hour patterns);
- the average of its road-neighbours' signals (what nearby traffic is doing).

We do not add “speed X minutes ago” features – the last hour is already in the input.

NN1 – split, normalisation, no leakage

Temporal (chronological) split: 60% train / 20% val / 20% test.

- train = past, test = future $\Rightarrow$ mimics real deployment.
- not random k-fold: autocorrelation would leak the future and inflate scores.

Per-sensor z-score normalisation using training statistics only (features and target); predictions denormalised back to mph for all metrics.

No look-ahead: a gap of $W = 12$ steps is dropped at each split boundary, so an input window never crosses into another split.

NN1 – architecture (GCN-GRU) & training

Spatial (per time step) – graph convolution:

$$H = \text{ReLU}(AXW_{\text{gcn}}), \quad A = \alpha A_{\text{fixed}} + (1 - \alpha)\tilde{A}, \quad \tilde{A} = \text{softmax}(\text{ReLU}(E_1 E_2^T)).$$

- Adaptive adjacency $\tilde{A}$: a learned graph (captures correlations geometry misses).
- Residual $H = \text{GCN}(X) + \text{Linear}(X)$: keeps each sensor's own signal.
- Temporal: 2-layer GRU over the 12 steps (shared across sensors).
- Head: $\text{Linear}(\text{hidden} \rightarrow 1) \Rightarrow$ one speed (60 min). No sigmoid.
- Training: Huber loss ($\delta = 1$), Adam (lr 10^{-3}), early stopping on val MAE, seed 42, $\sim 56\text{k}$ params, trained on Apple-Silicon GPU (MPS).

NN2 – predicting the whole interval $[+5, +60]$ min

Task: for every sensor, predict speed at all 12 steps 5, 10, ..., 60 min ahead.

- “Any segment at any near-future time” – the data lives on a 5-min grid.
- Target grows from $[170]$ to $[170, 12]$ (one value per horizon).
- Same 10 features, same 12-step input window as NN1.
- Standard benchmark setup (DCRNN, Graph WaveNet report 15/30/60 min jointly).
- Both models are kept in the code (flag `-single-horizon` selects NN1); NN2 is the default.
- Neither overwrites the other.

NN2 – what changes vs NN1

Same spatial GCN + adaptive adjacency + residual, same GRU, same split, normalisation, loss, training recipe.

Only the output head changes:

$$\text{Linear}(\text{hidden} \rightarrow 1) \implies \text{Linear}(\text{hidden} \rightarrow 12)$$

Output shape per sample: $[N] \Rightarrow [N, 12]$; Huber loss summed over all 12 horizons.

Predicting all horizons jointly acts as multi-task regularisation (one model, full curve).

Cost: $\sim 60\text{k}$ params (vs $\sim 56\text{k}$) – essentially free.

NN3 predicts later-today traffic speed

Near-future forecasting is useful for immediate control. We also predict farther ahead, where daily traffic rhythm becomes important.

Prediction horizons

+1h	+3h	+6h	+12h
12 steps	36 steps	72 steps	144 steps

Target

- one speed prediction per sensor
- four horizons predicted together

Leakage-safe inputs

- current flow, occupancy, speed at time t
- speed lags: 3h, 6h, 12h, 1d, 2d, 7d
- current and target
time-of-day/day-of-week

Built dataset

X	(15696, 170, 29)
y	(15696, 170, 4)
train / val / test	9417 / 3139 / 3140

No future traffic measurement is used as an input feature.

NN3 uses one shared GRU for all sensors

2 hours of features → shared GRU → +1h, +3h, +6h, +12h speeds

Why this architecture

- keeps the model small and stable
- learns temporal patterns shared by all sensors
- predicts all long horizons in one pass

Best run

- input window: 24 steps = 2 hours
- hidden size: 64
- GRU layers: 2
- dropout: 0.2
- parameters: 43,460
- loss: Smooth L1 / Huber
- device: CUDA GPU

Train-only normalisation is used, and metrics are reported after denormalising back to mph.

Results – NN1 vs NN2 vs baselines

Model (test set)	MAE	RMSE	MAPE
Persistence (avg over 5–60 min)	1.6133	3.7811	3.08%
Persistence (60 min only)	2.1403	4.9692	4.21%
NN1 – single-horizon (60 min)	1.7979	4.4583	4.13%
NN2 – multi-horizon (60 min)	1.7677	4.2927	3.98%
NN2 – multi-horizon (avg)	1.4897	3.6037	3.23%

At 60 min, NN2 matches/beats NN1 while also covering all shorter horizons.

NN2 beats the average persistence by +7.7% – and that average is hard (see next slide).

Results – the crossover (per-horizon MAE)

Placeholder

Per-horizon MAE crossover plot
Persistence vs. NN2
Use teammate figure when available.

Key finding

Persistence is near-perfect short-term (speed barely changes in 5 min).

The model overtakes at ~ 20 min and the gap grows: +17.4% at 60 min.

⇒ judge models per horizon, not by a single average.

Results – ablation (each component helps)

Placeholder

Ablation figure

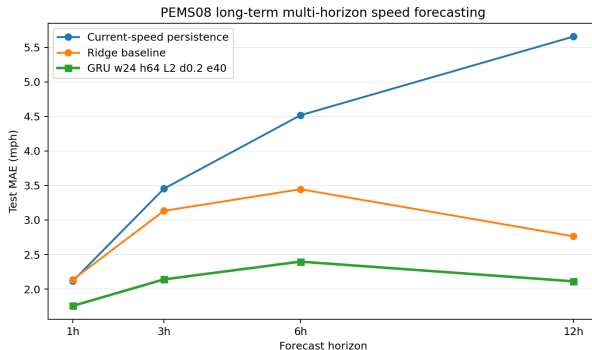
Fixed graph vs. adaptive adjacency vs. residual

Use teammate figure when available.

Fixed-graph base is weakest; + adaptive adjacency and + residual each lower MAE.

Same trend for NN1 and NN2 – the gains come from architecture, not scale.

NN3 result: lower error at every long horizon



Best GRU

Avg MAE: **2.1015**

Best baseline: ridge

- ridge avg MAE: 2.8699
- GRU improvement: 26.8%
- +6h improves by 1.0489 mph

Interpretation

The hardest point is +6h; +12h becomes easier because daily periodicity helps.

Takeaways

- PEMS08 is naturally a graph time-series forecasting problem.
- The short-horizon models predict immediate traffic dynamics.
- NN3 extends the project to later-today forecasting: +1h, +3h, +6h, +12h.
- Chronological splits and leakage checks make the reported scores realistic.
- The best long-horizon GRU beats the strongest baseline at every horizon.